

ATMT Assignment 5

Topic: improving decoding in NMT

Deadline: Tuesday, December 9, 2025, 23:59

Important submission info:

- You will have three weeks to complete this assignment.
- **Submission format: GitHub, pdf**
- Please name your submission files like this: `olusername_atmt_assignmentxx.pdf`, e.g. `mmuster_atmt_assignment05.pdf`
- For this assignment, you are asked to submit a report as a PDF file and your code and test set translations in a GitHub repository (with a link to the repository included in your PDF report). Please do not hand in any other files.
- You need to submit the assignment on OLAT using the tab for assignment 5. Please hand it in on time. The submission deadline is given above. After this time, the tab will be closed. We do not consider Git commits that happened after the deadline.
- Clearly and transparently indicate use of LLMs for this assignment.
- You are encouraged to work in groups of two. Please make sure every file clearly states **both** names. For pairs, only one person should submit the assignment.

1 Introduction

In this assignment, we focus on decoding, i.e. generating a word given a representation of the source sentence and the previously generated words. Until now, we have relied on a greedy search, which always chooses the single most likely next word to construct the output sequence. With such a method, we are not guaranteed to find the translation *sequence* with the highest overall probability. For example, at any given time step, choosing the most probable word may lead to a sub-optimal translation in terms of the overall probability of the sentence. Beam search is a very popular decoding strategy that offers an improvement over greedy search by allowing us to explore multiple translation hypotheses simultaneously. In this way, it enables the model to potentially recover from bad translation choices made in earlier decoding steps.

Beam search relies on a parameter k (typically called *beam size*) that defines how many translations are pursued simultaneously. At the first time step, instead of picking the word with the highest probability, we store the k words with the highest probabilities as different translation hypotheses. At every consecutive time step, we expand *all* of the k best hypotheses from the previous step and look at all of their probability distributions over the next word. We multiply the probability of a hypothesis with the probability of the possible next word to obtain the overall probability of this sequence (in practice, we use log-probabilities to avoid multiplying with very small numbers). From all of these possible new sequences, we pick the k sequences with the highest probabilities as our new hypotheses to expand in the next step, discarding all other (less likely) hypotheses up to this point.

The whole process is repeated until one of the k best hypotheses ends with an end-of-sentence symbol. Then we consider this as one of our k final options and continue generating larger sequences with the other $k - 1$ hypotheses until all end with the end-of-sentence symbol. Finally, we choose the hypothesis with the highest overall probability as our final translation. In this way, beam search gives a good trade-off between translation quality and decoding time. In practice, there may be some differences to this theoretical description, e.g. we run multiple beam searches for every element in a batch at the same time to keep the efficiency of matrix operations.

In the updated [atmt repository](#), there is a new file `translate_beam.py` which contains an implementation of beam search. The following tasks should be solved by modifying and utilizing this implementation. As beam search is purely a decoding mechanism, you do not need to train a new NMT model for this. You are free to choose from any of your previously obtained baselines. For this task, you may also share baselines among students/assignment groups¹. **Please ensure that you clearly indicate which baseline you use, as not doing so may result in deduction of points!**

¹There has been relatively high variance in terms of BLEU scores between different seeds, thus if you trained a successful model (20-30 BLEU), feel free to share it with others.

2 Understanding the Code

2.1 Code Questions

In the later sections of this assignment, you will need to alter parts of the decoding. This may be difficult if you are not familiar with the code. To help you understand how the code works, there are six questions in `translate_beam.py` that you need to answer. The questions are marked with `# __QUESTION X:`. Go through the code carefully and try to understand it before answering the questions. You should be able to answer every question in a few sentences. The description of beam search in the previous section should be a good starting point. Sometimes it may also be useful to look outside of `translate_beam.py` to understand the code (e.g. look at what is returned by the encoder or the decoder).

Submission: Enumerate your answers to these questions in your PDF submission file.

2.2 Beam Search Variants

This paper ² distinguishes between two variants of beam search, referred to as *Vanilla* and *FCFS* (*First Come, First Serve*). The differences are outlined in section 2 of the paper.

Submission: Look through the code of `beam_search_decode()` and identify the variant. Explain how it may differ from its counterpart in terms of how the best hypothesis is obtained.

3 Optimizing the code

Currently, both the original (greedy) decoding function and the beam search decoder are inefficient. Find out why, implement the changes (in both functions), and measure the difference in decoding time. It should not take more than changing a few lines of code!

Hint: An expensive computation that could be done outside of a for-loop is executed repeatedly inside of it. The problematic line of code does not pose the same inefficiency issues during training.

Submission: Explain what the problem was, how you fixed it, and report the difference in decoding speed for varying beam sizes (e.g. $k = 1, 3, 5$) on a **small** sample (< 100 sentences).

4 Improving Beam Search

4.1 Length Penalty

Currently, the hypotheses are scored solely by their accumulated log-probabilities, without any length normalization. There is already a parameter α (**alpha**) in the translation script that

²<https://arxiv.org/abs/1702.01806>

is passed to `beam_search_decode()` but unused as of yet. Implement the following length normalization (taken from the slides of week 2):

$$score(Y, X) = \frac{\log(P(X|Y))}{\frac{(5+|Y|)^\alpha}{6^\alpha}}$$

For a small subset of sentences, explore how the average length of the output translations changes for various values for α (with $\alpha = 0$ being no length normalization).

Submission: Explain the rationale behind length normalization. Describe the changes to the code and report/interpret the results you got.

4.2 Exploring Stopping Criteria in Beam Search

Beam search is a common component in NMT sequence generation tasks. The stopping criterion used during beam search can significantly impact both the quality of the generated translations and the efficiency of the decoding process. In this task, you will explore different stopping criteria and analyze their effects. There are various stopping criteria used in different beam search implementation. [This paper](#)³ provides a good overview.

First, you are asked to examine the function `beam_search_decode()` in `decode.py`, and find out where the current (constant) stopping criterion is implemented.

Next, take a look at the alternative stopping criteria proposed in said [paper](#) (section 4, *Search Strategies*). Pick 2 that you find interesting, and create two alternative beam search decoding functions that each implement one of them.

Submission: Describe how you implemented each of the two stopping criteria you chose. Analyze for each how this stopping criterion affects translation quality and decoding efficiency. Provide examples where this stopping criterion led to different translations.

5 Final remarks

Please also push your code changes to your GitHub repository and include the link in your PDF report. Don't forget to report which baseline you decide to use, and indicate your usage of LLMs.

Please let us know in the OLAT forum if you have problems or something is unclear.

Good luck!

³<https://arxiv.org/abs/1702.01806>